Advik: handbook

System, evaluation, deep search, live viewer (lead)

Read 00_Start_Here first; then this handbook.

Machine Learning (702DB0C015), B.Tech CSE (Data Science), Semester V, 2026-27

Contents

1. Part brief
2. Knowledge pack
3. Presentation guide
4. Viva questions and answers
5. Diagrams

Advik: system architecture, evaluation, deep search, live viewer (LEAD)

Marks this part carries: Model Evaluation and Performance (3), Documentation and Presentation (2), and the integration that makes the other parts one system. **Why it is the most important part:** every number the team quotes comes from the evaluation you own; it contains the main new method (the deep search); and it holds the pieces together. You also open and close the presentation.

Your four jobs

1. **The system as a whole**: the architecture, how the stages connect, the deployed path `run_night_pipeline`, model bundles (versions), configs, tests, reproducibility.
2. **The evaluation framework**: planting fakes, the scoreboard, held-out nights, fair model comparison, what the numbers mean, and the negative results (a multi-frame CNN tied XGBoost; 3x more data did not help).
3. **The deep search** (shift-and-stack): the main method that goes fainter than a single frame, with its statistics (look-elsewhere effect, null test) and its measured depth.
4. **The live viewer and the demo**: the interface that runs the real pipeline stage by stage; plus a short account of fast movers and the live monitor.

What is NOT yours (hand over politely)

- How the data is downloaded and preprocessed: Aaryan.
- Labels, stamps and the CNN: Paras.
- Linking, tracklet features, XGBoost, metric definitions: Satvik. You must understand each in outline (see the cross-part cheat sheet in your `KNOWLEDGE_PACK.md`), because you integrate them.

Where your code is (in `project/`)

Area	Files	What they do
Evaluation	`src/asthunt/evaluate/build_scoreboard.py`, `save_scoreboard_rows.py`, `measure_completeness.py`, `find_magnitude_at_completeness.py`, `measure_depth_at_thresholds.py`, `measure_injection_realism.py`	the headline numbers from one function; completeness curves; depth vs false alarms; "are our fakes realistic?"
Deployed path	`src/asthunt/pipeline/run_night_pipeline.py`, `run_full_search.py`, `run_deep_search.py`, `run_fast_search.py`, `run_live_monitor.py`	the single code path used for evaluation *and* the real search
Model versions	`src/asthunt/models/save_model_bundle.py`, `load_model_bundle.py`; `models/v1 ... v5`	a version = CNN + tracklet model + thresholds + provenance

Area	Files	What they do
Multi-frame experiment	`models/define_multiframe_model.py`, `train_multiframe_model.py`, `predict_multiframe.py`; `features/add_path_features.py`	the model that tied XGBoost; the two physics features that were kept
Deep search	`src/asthunt/deep/*.py` (11 files): `build_snr_image`, `standardize_over_night`, `group_frames_into_stacks`, `make_velocity_grid`, `stack_and_find_peaks`, `mask_bright_pixels`, `merge_peaks`, `measure_velocity_consistency`, ...	matched filter, per-pixel standardisation, GPU stacking over 4,888 motions, peak merging, consistency cut
Fast movers	`src/asthunt/fast/*.py`	trailed-streak matched filters and the trail-vs-motion consistency check
Live viewer	`src/asthunt/viewer/*.py`, `scripts/live_viewer.py`	web page + server + a traced run of the real functions, one stage at a time
Tests	`tests/evaluate`, `tests/pipeline`, `tests/deep`, `tests/fast`, `tests/viewer`	
Lessons and notebooks	`docs/scoreboard.md`, `multiframe_model.md`, `more_data.md`, `deep_search.md`, `fast_movers.md`, `live_monitor.md`, `full_run_mlops.md`, `label_quality.md` (evaluation half), `docs/architecture/`; the notebooks with the same names	

What you hand to the team

- One list of numbers with their source files (start from `05_Team_Plan.md`) so slides and report agree.
- The demo: a laptop where `python scripts/live_viewer.py` works on a night with fakes.
- The report chapters on evaluation methodology, the deep search and limitations.

Your seven key messages (say these even if you forget everything else)

1. **We do not trust a number unless it comes from the deployed pipeline on nights nobody tuned on.**
2. **v1 looked perfect (tracklet AP 0.999) but the scoreboard said depth only ~20.5.** The right metric was depth and recovery of known asteroids.
3. **We found where asteroids were lost (linking), fixed that, and a bigger model only tied a simple one.** Use the simplest model that ties.
4. **More data did not help, because one 30 s frame holds only so many photons.** Adding frames (the deep search) did: +0.7 magnitude.
5. **The deep search needed statistics, not just code:** 4,888 trials means noise reaches 5 sigma, so we need 6 sigma, and a null test with shuffled times gave 0 false tracks.
6. **Compare models at the same false-alarm level.**
7. **No discovery is claimed.**

Numbers you must know by heart (with where they come from)

Number	Meaning	Source
20.5 -> 21.4 -> 22.1	50%-completeness magnitude: v1, v4, deep search (test night 2019-02-08)	`reports/scoreboard.csv`, `docs/deep_search.md`
4,888	trial motions in the deep search	`docs/deep_search.md`
6,179 -> 44	false peaks per stack at 5 sigma vs 6 sigma on pure noise	`docs/deep_search.md`, `configs/deep.yaml`
47,550 -> 94	peaks per stack from first version to final	`docs/deep_search.md`
16 -> 0	null-test false tracklets per night before/after the velocity-consistency cut	`configs/deep.yaml`
20 of 20	known asteroids kept by that cut	same
0.7 mag	gain of deep search over best single-frame system on the same night	`docs/deep_search.md`
1,499	chance tracks per night with relaxed linking	`docs/multiframe_model.md`
0.9933 / 0.9924	multi-frame CNN vs XGBoost average precision (a tie)	`docs/multiframe_model.md`
150+	offline tests	`pytest`

Hands-on tasks (do these yourself; they make the explanation yours)

1. `cd project && pytest -q` (all pass).
2. `python scripts/live_viewer.py`: run a night with 8 fakes, step by step; note which fakes are missed and why.
3. Re-run `notebooks/scoreboard.ipynb` and `deep_search.ipynb`; write one paragraph each in your own words.
4. Change `configs/deep.yaml: peak_threshold_sigma` from 6.0 to 5.0 in a copy and predict (then check) what happens to the false-peak count.
5. Open `reports/scoreboard.csv`; for each of v1, v2, v4, v5 find the test-split depth and unexplained alarms per night and explain the pattern in two sentences.

Pitfalls

- Quoting a depth number without saying *which night and quadrant*.
- Saying "the model is 99.9% accurate": it is a saturated metric on a planted-fake-heavy set.
- Claiming the deep search "finds new asteroids": it reaches fainter *known* and unknown objects; no discovery is claimed.
- Forgetting that the test night was looked at while fixing real bugs (the holdout night exists because of that).

Advik: knowledge pack

0. Read first (in this order)

`00_Start_Here/01_Project_Overview` -> `02_Domain_Primer` -> `03_ML_Primer` (sections D and E especially) -> then the lessons `project/docs/scoreboard.md`, `multiframe_model.md`, `more_data.md`, `deep_search.md`, `fast_movers.md`, `live_monitor.md`, `full_run_mlops.md`.

---

1. The system as a whole (architecture and engineering)

The pipeline (see `diagrams/architecture_full.svg`): data -> difference images -> detect blobs -> stamps -> CNN -> link -> features -> XGBoost -> cross-match -> candidates. `run_night_pipeline(unit, metadata, configs, bundle, paths, rng, ...)` runs those stages in order and returns detections, members, tracklets, candidates, injections, known objects.

One deployed path. The same function is used (a) with fakes planted, for evaluation, and (b) without, for the real search. *Why it matters:* notebooks that re-implement pipeline steps drift from the real pipeline, so their numbers stop describing what you would run. The scoreboard calls the same function the live system calls.

Bundles (a mini model registry). `models/v4/` contains the CNN, the tracklet model, `thresholds.json` (CNN cut 0.0081, tracklet cut 0.601, minimum 3 detections) and `bundle.json` (git commit, training nights, the 17 feature names). *Why together:* mixing a new CNN with an old threshold is a classic production bug. Bundles are never overwritten, so you can roll back.

Configs, not constants. Every setting lives in `configs/*.yaml` (night split, thresholds, sizes). *Why:* a result must be tied to its settings; the split is written before modelling so it cannot quietly change after disappointing results.

Tests. About 150, all offline (they build tiny fake images). Important ones: the end-to-end pipeline test (3 tiny frames with pointing shifts; fakes linked, never reported as discoveries), the leakage test, the viewer test (the traced run equals the pipeline).

Provenance. Which code (git sha in model names), settings (configs), software (`environment.yml`), data (cached archive tables), random fakes (seed = [project seed, night]), thresholds (bundle).

---

2. The evaluation framework (your most important topic)

2.1 Injection-recovery and completeness

We cannot know the true asteroids in an image, so we **plant** 300 fakes per night (brightness 17-22, speed 5-100 arcsec/hour, any direction), run the *normal* pipeline, and count how many come out. The recovered fraction versus brightness is the **completeness curve**; the magnitude where it crosses 50% is the **depth**.

Fakes are made realistic on purpose: the real PSF of each frame, Poisson noise, added to both science and difference images, real frame times, and, after a bug fix, moving straight **on the sky** (not in each frame's pixels). **Realism check (adversarial validation):** a classifier asked to tell fakes from real known asteroids (magnitude-matched, cross-validated by night) gets AUC 0.46-0.48; 0.5 means indistinguishable.

2.2 The scoreboard

One function, `build_scoreboard`, computes: detection AP and recall, bogus kept per frame, tracklet AP, **depth at 50% completeness**, **known asteroids recovered**, **unexplained alarms per night**. Results are saved to `reports/scoreboard.csv`, one row per (version, split); re-running replaces rows. *Before the scoreboard:* every notebook computed numbers slightly differently (the same baseline showed 813 and 804 false positives per frame in two lessons). *After:* one definition.

split	v1 depth	v4 depth	v1 known recovered	v4 known	v4 unexplained/night
validation (6 units)	20.18	20.49	21/28	24/28	0.0
test (3 units)	20.53	21.34	39/44	40/44	5.0
holdout (3 units)	20.14	20.47	16/18	17/18	0.3
(from `reports/scoreboard.csv`; the single-quadrant test-night numbers quoted elsewhere are 20.51 and 21.41.)					

2.3 Holdout and honesty

Nights are split train / validation / test **by night** in `configs/acquire.yaml`. A **holdout** night (2022-06-17) is opened only by the scoreboard. *Honesty note:* the test night was looked at while fixing real bugs (pointing shift, units, speed noise). Those were physics corrections, not score tuning, but strictly the test night is no longer untouched; that is why the holdout exists and why the holdout numbers match the validation numbers (a good sign nothing was secretly tuned).

2.4 Comparing models fairly

- **Same operating point.** At each model's own threshold, v5 looked 0.8 mag shallower on test than v4 (20.56 vs 21.34). At matched false-alarm levels (about 10, 5, 3 alarms per night) v4 was equal or slightly better (21.38 vs 21.28, 21.34 vs 21.25, 21.33 vs 21.09). The "drop" was mostly v5's threshold landing at a stricter point on the same trade-off curve.
- **Metrics with headroom.** Tracklet AP is about 0.999 for every version: saturated, cannot show progress. Depth and known-asteroid recovery have headroom.
- **Measure where the loss is before building a model.** On one training night: 95% stage-1 recall with 4-of-5 linking gave 184/294 fakes linked and 9 bogus tracklets; 3-of-5 gave 200/294 and 74 bogus; 99% recall gave 205/294 and 1,499 bogus. The loss was in linking, not in the classifier.
- **A tie means the simpler model.** Multi-frame CNN vs XGBoost (AP all: 0.9921 vs 0.9924; averaged 0.9933): a tie, so keep XGBoost.

2.5 Negative results (say them with pride)

- A fancy model did not beat a simple one.
- More data did not help.
- The v2 system raised 15 unexplained alarms on the dense test night; looking at all 15 by eye showed artifact walkers and noise alignments, which led to two physics features (static neighbours, forced photometry) that cut alarms from 15 to 7 on the single-quadrant test night (13.3 to 5.0 on the three-quadrant scoreboard).
- A bug in your own feature: it counted a slow asteroid's own detections as "static neighbours", flagging 22% of real tracklets; found by checking the feature on real examples; fixed (0.4%); a test was written; v3, trained with the buggy feature, was retired and v4 trained correctly.

---

3. The deep search (shift-and-stack)

3.1 The idea and the maths

A faint asteroid is below the noise in any single 30-second frame. If you know its motion you can shift each frame so it sits on the same pixel and add them. Signal adds N times; independent noise adds sqrt(N) times. 8 frames: gain 8/2.83 = 2.83x in SNR = 2.5 x log10(2.83) = **+1.13 magnitudes**.
(`diagrams/shift_and_stack.svg`)

3.2 We do not know the motion, so we try 4,888 of them

Every direction, up to 120 arcsec/hour, in steps of 3 pixels/hour (a half-step over a 0.5-hour stack is a worst-case smear of 0.75 px). On the Mac's GPU. Then we **link** stack detections across the night (a real object must appear in several independent stacks along one straight path): 55 frames over 4.4 hours -> 7 stacks of about 8 frames.

3.3 The statistics (the part examiners love)

- **Look-elsewhere effect.** Picking the best of 4,888 motions at every pixel makes chance peaks common. On pure-noise stacks: 6,179 false peaks at 5 sigma but only 44 at 6 sigma, so the cut is **6 sigma** (set from data, not assumed).
- **Real data are not Gaussian.** The first version at 6 sigma still gave about 47,000 peaks per stack. Each fix and its effect: clip each frame at 5 sigma (one bright frame cannot dominate); subtract the night's median per pixel (static residuals no longer stack at any motion) -> 5,765; **per-pixel standardisation** by median *and scatter* (residuals that wobble with the seeing are turned down) -> 391; **mask pixels single frames already see** (otherwise bright asteroids create echo tracklets) -> **94**.
- **Echo test and null test.** Before masking, 151 of 242 unknown tracklets lay within 30 px of a known asteroid (random would be about 190 px away): ghosts. A **null test** (frame times shuffled, so real movers no longer line up) gave 16 false tracklets per night; a **velocity-consistency cut** (each stack's own best motion must match the night-long motion within 6 px/h: known asteroids 2.4 px/h, null tracklets 61 px/h) took the null to **0** while keeping **20 of 20** known asteroids.
- **Calibration:** mag = 26.25 - 5.23 log10(mean stack SNR), scatter 0.17 mag, fitted on 100 found fakes (a straight-line regression).

3.4 Result (300 injected fakes, magnitude 20-23, test night)

50% completeness at **22.12** vs 21.41 for the best single-frame system on the same night (+0.7 mag). Recovery: 94-100% at 21.4-21.9, 55% at 21.1 (masked on purpose below ~20.9 because single frames already see them), 48% at 22.1, 0% at >= 22.4. *Limits:* needs at least 20 frames in a night (8 + 8 + 4 = three stacks); a night with 18 frames silently found nothing until we added the check. It was developed on one night/quadrant.

---

4. Fast movers and the live monitor (brief)

- **Fast movers:** near-Earth objects at 300-15,000 arcsec/hour trail during a 30 s exposure (length in px about rate / 120). A matched filter for trails (4-32 px, every 15 degrees); a track is kept only if the trail points along the motion (within 15 degrees), its length equals the speed x exposure (ratio 0.625-1.6) and the length is steady. 97% of planted trails recovered on the nights used to choose the cuts (cuts chosen on the same nights: a stated limit); 93% on a later 10-frame night; 0 false tracks. Real-data lesson: six of seven fast "tracks" we inspected ran along CCD columns (bleed-trail residuals); a veto exists as an option in `configs/fast.yaml`, off by default, costing about 1 point of recovery.
- **Live monitor:** watches the archive, runs point + fast + deep searches on a complete night, remembers what it has done (idempotent), alerts on unknown objects; replay of a past night gave 1 alert and no duplicates on re-run.

---

5. The live viewer (your demo)

`python scripts/live_viewer.py` starts a small web server and opens a page. Choose a night, set "fakes", press Start (tick "step by step" to pause after every stage). It runs the **real** functions in the real order and draws each stage: the four files, the noise histogram, planted fakes, blobs (with a zoom), stamps, CNN verdicts, tracks, features, classifier scores, known-asteroid matches, the result. A test proves that on a real night the traced run equals `run_night_pipeline` exactly (max difference 0.0). Design in `diagrams/live_viewer_flow.svg`; code in `src/asthunt/viewer/`. *Demo tip:* use 8 fakes; the bright ones are found and the faint ones are missed, which shows the depth limit live.

---

6. Cross-part cheat sheet (you must be able to say these)

- **Aaryan (data):** ZTF difference images; unit = field+CCD+quadrant+night; downloads are resumable and atomic; split by night; mask bad pixels; noise via MAD; leakage audit (accuracy 100% -> 50% -> 99%).
- **Paras (labels + CNN):** labels by injection, known asteroids (Horizons 0.26 px vs SkyBoT 2.7 px) and "the rest"; 3x31x31 stamps normalised by MAD; CNN with 5 conv layers, 72,145 weights, dropout, early stopping; AP 0.899 vs baseline 0.780; cross-fitting by night.
- **Satvik (linking + trees + metrics):** linking by seed-and-predict with KD-trees on the common sky grid; 17 features; XGBoost (300 trees, depth 3, lr 0.05); models tie in cross-validation; accuracy paradox, PR vs ROC, thresholds chosen on validation, calibration.

7. Self-quiz (answers in `VIVA_QA.md`)

1. Why does the scoreboard call `run_night_pipeline` instead of reading saved tracklet files? 2. Why must thresholds live in the same bundle as the models? 3. Why is tracklet AP 0.999 a poor metric for v2? 4. What would stop the holdout night being a holdout? 5. Explain the +1.1 magnitude of an 8-frame stack.
2. Why 6 sigma and not 5? 7. What does the null test prove, and what does it not? 8. Why did v5 look worse and why was that misleading? 9. What is the difference between a null test and an injection test? 10. How does the viewer prove it shows the truth?

Advik: presentation guide

You speak **twice**: the opening (3 min) and the main block at the end (5 min + demo). Total about 8-9 minutes. If the slot is shorter, see the 12-minute plan at the end.

Part A: the opening (3 minutes, 3 slides)

Slide A1: "What we built" (45 s)

On the slide: title; one sentence; the picture `00_Start_Here/diagrams/system_overview.svg` is for A2. **Say:** "Telescopes photograph the same sky again and again. Stars stay still; asteroids move. We built a machine-learning pipeline that finds moving objects in old, public images from the Zwicky Transient Facility, tells real ones from junk, and, most important for this course, measures honestly how well it works."

Slide A2: "The pipeline and the team" (1 min 30 s)

On the slide: `system_overview.svg` (colour = owner). **Say:** "Images come from the ZTF archive. Aaryan will show how we prepare the data. Paras will show the neural network that decides which blobs are real. Satvik will show how blobs become tracks, how a tree model scores them, and the metrics. I will show how we evaluate everything, a deep search that goes fainter than any single frame, and a live demo. Two models learn: a CNN on pixels and gradient-boosted trees on track numbers. Everything else is physics, search and statistics."

Slide A3: "The question we ask" (45 s)

On the slide: two bullets: "How deep can we see?" and "Can we trust the number?" **Say:** "Accuracy is useless here: only 3% of blobs are real, so a model that says 'bogus' every time is 96.8% accurate and finds nothing. So we ask two harder questions: how faint an asteroid can we still recover, and how do we know that number is honest? The rest of the talk answers both." **Transition:** "First, the data. Aaryan."

Part B: the main block (5 minutes + 3 minute demo)

Slide B1: "How we test: the evaluation protocol" (1 min)

On the slide: `diagrams/evaluation_protocol.svg`. **Say:** "We plant fake asteroids of known brightness into the real images, run the same pipeline we would use for the real search, and count what comes out. Thresholds are chosen on validation nights, never on test. One function, the scoreboard, produces every number. And there is a holdout night that only the scoreboard opens. When we compare models we compare at the same false-alarm level, because each model's own threshold lands on a different point of its trade-off curve."

Slide B2: "What the scoreboard taught us" (1 min 15 s)

On the slide: `diagrams/scoreboard_versions.svg`. **Say:** "Version 1 looked perfect: the tracklet classifier had average precision 0.999. The scoreboard said otherwise: depth only about 20.5 magnitudes. Bigger magnitude means fainter. We measured where asteroids were being lost and it was not the classifier, it was linking. Relaxing linking found 11% more asteroids but made about 1,500 chance tracks per night; two physics features controlled them, giving 21.4. A bigger multi-frame network only tied the simple XGBoost, so we kept the simple one. Three times more training data did not help either, because one 30-second frame holds only so many photons."

Slide B3: "The deep search" (1 min 30 s)

On the slide: `diagrams/shift_and_stack.svg`. **Say:** "If you cannot see an asteroid in one frame, add frames. Shift each frame along a guessed motion so the asteroid lands on the same pixel and add them: the signal grows N times, the noise only the square root of N. Eight frames give plus 1.1 magnitudes. We do not know the motion, so we try 4,888 of them on the GPU. That creates a statistics problem: pick the best of thousands of trials and noise alone looks significant. On pure noise we saw 6,179 false peaks at 5 sigma and only 44 at 6 sigma, so we require 6 sigma. A null test, with the frame times shuffled so real movers cannot line up, produced zero false tracks, while 20 of 20 known asteroids survived. Result: 50% completeness at magnitude 22.1, 0.7 magnitudes deeper than our best single-frame system on the same night."

Demo (3 minutes): the live viewer

See the demo script below.

Slide B4: "Limits and conclusion" (45 s)

On the slide: four bullets: limits and the one-line conclusion. **Say:** "Limits: the best depth is for one night and one chip quadrant; our real labels come partly from simulation; the test night was looked at while fixing real bugs, which is why we also keep a holdout night; and 'unknown' only means 'not in the catalogues we queried'. We do not claim a discovery. What we do claim is a pipeline whose numbers you can trust, a clear picture of where the limit is, and honest negative results: a bigger model tied a small one, and more data did not help."

Demo script (3 minutes, rehearse twice)

Setup: laptop with the data; `python scripts/live_viewer.py` already running; browser on `http://127.0.0.1:8765`; choose a **training** night (e.g. 2020-09-19, field 618, CCD 3, quadrant 4); fakes = 8; "step by step" ticked.

1. **Start.** "The agent now picks the night." Press Next. *Stage 2:* "It opens four files per exposure. Look at the third panel: the difference image, where stars have cancelled."
2. **Noise.** "The histogram should be a bell curve if there were only noise. The tail is the real blobs and artifacts."
3. **Plant fakes.** "These 8 fakes are our test objects, brightness and speed known."
4. **Find blobs.** "Seven thousand blobs in one frame, nearly all junk. The zoom shows one planted fake (teal ring) among noise."

5. **CNN stage.** "The network gives each blob a probability. The histogram shows junk piled near zero and our fakes near one." Show the "most convinced it is bogus" mosaic quickly.
6. **Link.** "Blobs that line up across frames become tracks; the close-ups show the straight lines."
7. **Result.** "6 of 8 fakes are recovered; the two faint ones (magnitudes 21.5 and 21.7) are missed. That is the depth limit you saw in the scoreboard. And no unexplained candidate: everything found is a known asteroid or a fake." Switch to automatic (untick step by step) if time is short; the whole run takes about 20 seconds. **If the demo fails:** show the pictures in `01_Advik/demo_backup/` (already saved: one per stage, plus the finished page) and say "the page runs the real pipeline functions; here is a recording of the run."

Anticipated first questions (prepare 20-second answers)

1. "Did you find a new asteroid?" -> No; we recover known ones to about magnitude 21 and have no discovery claim.
2. "Why not just use accuracy?" -> 3% positives; accuracy paradox.
3. "How do you know the fakes are realistic?" -> AUC 0.47 when a classifier tries to tell them from real asteroids.
4. "Why does more data not help?" -> limited by photons in one frame, not by model size.
5. "What is the look-elsewhere effect?" -> picking the best of many trials; hence 6 sigma.

The 12-minute plan (if your slot is short)

Opening: 1 slide, 1 minute (pipeline picture + "how deep and can we trust it"). Main block: B2 (1 min), B3 (1.5 min), a 30-second demo clip or two screenshots, B4 (30 s). Skip B1 but mention "we compare at equal false alarms" in B2.

Delivery tips

- Speak to the diagram, not the screen text. Point at the box you are describing.
- Use "bigger magnitude = fainter" the first time you say a magnitude.
- Give *one* number per sentence. Round (20.5, 21.4, 22.1) and name the source if asked.
- If you do not know: "I did not test that; I would check it by ..."

Advik: viva questions and model answers

Answer in your own words; the answers below are the skeleton. Sections: A system, B evaluation, C deep search, D fast/live/viewer, E the whole project and ML in general, F tricky ones.

A. System and engineering

1. Why does the scoreboard call `run_night_pipeline` instead of reading saved tracklet files? To measure exactly what would be deployed. Notebooks that re-implement steps drift from the real pipeline, and then their numbers describe something nobody runs. One path, used with planted fakes for evaluation and without for the real search.

2. Why must the thresholds live in the same bundle as the models? A threshold is only meaningful for the model that produced the scores. Mixing a new CNN with an old cut is a classic production bug, so a version = CNN + tracklet model + thresholds + commit + nights, never overwritten, so we can roll back.

3. What does a "bundle" contain and how is a version chosen? `models/v4/`: `realbogus_cnn.pt`, `tracklet_classifier.json`, `thresholds.json` (CNN 0.0081, tracklet 0.601, min 3 detections), `bundle.json` (commit, train and validation nights, the 17 feature names). `configs/live.yaml` names the bundle in use (v4).

4. How is the project reproducible? Settings in `configs/`; night split in a config; seeds `[project seed, night date]`; archive answers cached to CSV; known-asteroid lookups cached; `environment.yml`; about 150 offline tests; model names carry the git sha.

5. What do the tests protect? Give two examples. The end-to-end pipeline on tiny fake frames with pointing shifts (fakes must link and never be reported as candidates), the leakage check (no night in two splits), and the viewer test (traced run equals the pipeline). Many tests reproduce a bug we actually had, e.g. the static-neighbour feature counting a slow asteroid's own detections.

B. Evaluation

6. What is injection-recovery and why is it the standard method? Plant objects of known properties, run the normal pipeline, count what comes out. It gives labels and completeness (recovery vs brightness/speed). The sky is not labelled; our own fakes are. Weakness: fakes are only as realistic as the simulation, so we test the realism (AUC 0.46-0.48 when a classifier tries to separate fakes from real known asteroids).

7. What does "depth" or "50% completeness at magnitude 20.5" mean? Half of the planted fakes at that brightness are recovered. Bigger magnitude = fainter, so deeper means a larger number.

8. v1 had tracklet AP 0.999. Why is that a poor metric for judging v2? It is saturated: 96% of tracklets in that data are real (we plant 300 fakes per night), the opposite of the real sky. A saturated metric cannot show improvement. Depth, known asteroids recovered and unexplained alarms per night have headroom.

9. What would make the holdout night stop being a holdout? Looking at it to choose a setting, a threshold, a feature or a model. It is opened in one place only, to write one number. Honesty: the test night was looked at while fixing real bugs, which is why a separate holdout exists.

10. v5 (more training data) had depth 20.56 on test vs 21.34 for v4. Is v5 worse? Not really. Each model's threshold is chosen by a rule ("95% recall on validation") that lands at a different point on its own trade-off curve; v5's landed at a much stricter point (0.3 alarms per night vs 5). At matched alarms (about 10, 5, 3 per night) v4 was equal or slightly better (21.38 vs 21.28; 21.34 vs 21.25; 21.33 vs 21.09). Compare at the same operating point.

11. Why did more data not help? The system is capped by what a single 30-second frame can show (about 21.4 on a good night), not by the model. More examples cannot reveal photons that are not there. Data helps when the model is the bottleneck; here linking and the frame depth were.

12. How did you decide the problem was linking, not the classifier? Before building a model we measured where planted asteroids were lost: with 4-of-5 linking, 184 of 294 fakes were linked; with 3-of-5, 200; at 99% stage-1 recall, 205 of 294 (but 1,499 bogus tracklets). Relaxed linking finds 11% more.

13. A multi-frame CNN scored AP 0.9921 and XGBoost 0.9924. What do you do? It is a tie, and the 3-detection subset has only 33 real tracklets so its differences are noise. Use the simplest model that ties: XGBoost. (Average of both: 0.9933, a negligible gain.)

14. What went wrong with v2 and how did you fix it? On the dense test night v2 raised 15 unexplained alarms. We looked at all 15 by eye: "artifact walkers" (slow tracks along a faint line present in every frame) and "noise alignments" (three 2-sigma blips with nothing at the predicted spot in other frames). We added one physics feature for each failure mode: static neighbours and forced photometry. Alarms fell to 7 on that night (v4).

15. You found a bug in your own feature. Tell the story. The static-neighbour feature counted detections left behind at a tracklet's own positions in other frames, but slow asteroids leave their own detections nearby, so it flagged 22% of real tracklets. I checked the feature on real examples before trusting the rule, fixed it (0.4%), added a test that reproduces the bug, retired v3 (trained with the buggy feature) and trained v4 correctly. Lesson: check a feature on real examples; do not retrain old versions with new code silently.

C. Deep search

16. Explain shift-and-stack and the +1.1 magnitude. Shift frames along a trial motion so a moving object lands on one pixel and add. Signal adds N times; independent noise adds sqrt(N) times. N = 8: 8 / 2.83 = 2.83x in SNR. Magnitude gain = 2.5 log10(2.83) = 1.13.

17. Why 4,888 trial motions? We do not know the motion. We cover every direction up to 120 arcsec/hour in steps of 3 px/hour (a half step over a 0.5 h stack is a 0.75 px worst-case smear). The GPU makes it feasible.

18. What is the look-elsewhere effect and how did you handle it? Testing many places or motions makes chance peaks likely (picking the best of 4,888). On pure-noise stacks, 5 sigma gave 6,179 false peaks and 6 sigma only 44, so the threshold is 6 sigma, set from data.

19. Real data are not Gaussian. What went wrong first and how did you fix it? At 6 sigma, real frames still gave about 47,000 peaks. Fixes in order: clip each frame at 5 sigma (no frame dominates); subtract the night's median per pixel (static halos and galaxies no longer stack at any motion) -> 5,765; standardise per pixel by median and scatter (residuals that wobble with the seeing are turned down) -> 391; mask pixels single frames already see (otherwise bright asteroids make echo tracklets) -> 94.

20. What is a null test and what does it show? Repeat the whole search with the frame times shuffled so real movers no longer line up; whatever it finds is a false alarm. We got 16 false tracklets per night before the velocity-consistency cut and 0 after, while 20 of 20 known asteroids were kept. It shows the false-alarm rate of the method on that data; it does not prove any single candidate is real.

21. What is the velocity-consistency cut? Each stack's own best motion must agree with the night-long motion within 6 px/hour. Known asteroids agree to 2.4 px/h; null tracklets differ by 61 px/h.

22. What does the deep search deliver and what are its limits? 50% completeness at magnitude 22.12 vs 21.41 for the best single-frame system on the same night (+0.7). Limits: needs at least 20 frames in a night (three stacks); built and measured on one night and quadrant; fakes brighter than about 20.9 are masked on purpose because single frames already see them.

23. How do you convert stack SNR to magnitude? A fitted straight line: mag = 26.25 - 5.23 log10(mean stack SNR), scatter 0.17 mag, from 100 found fakes. (A small regression.)

D. Fast movers, live system, viewer

24. Why a separate fast-mover search? Near-Earth objects move 300-15,000 arcsec/hour, so they trail (about rate/120 pixels in 30 s) and a round-blob detector under-counts them. A matched filter for trails plus a consistency check (trail points along the motion within 15 degrees; length equals speed x exposure) recovers 97% of planted trails on the tuning nights, 93% on a later night.

25. What did you learn from real fast tracks? Six of seven we inspected ran along CCD columns: bleed-trail residuals. A veto exists but is off by default because it was chosen on the same tracks (optimistic) and costs about 1 point of recovery.

26. How does the live viewer prove it shows the truth? It calls the same functions in the same order as the pipeline. A test on tiny frames, and a run on a real night, show identical detections, scores, tracks and candidates (max difference 0.0).

27. What would you change to run it in production? `run_live_monitor`: poll the archive, wait until a night is complete, run point + fast + deep, store results idempotently, alert on unknown objects. A person reads every alert.

E. Big picture

28. What is the single most important lesson of the project? Measuring honestly beats modelling cleverly: the scoreboard exposed that a perfect-looking metric hid a depth problem, the simplest model tied the complex one, and the biggest gain came from physics (adding frames), not from more learning.

29. What would you do with another month? Evaluate the deep search on more fields; add the SVM and clustering experiments the syllabus mentions; a learning-curve study; independent confirmation from another survey for any object we cared about.

30. If we gave you only 60 seconds for the whole project? "A pipeline that finds moving objects in telescope images. A CNN removes junk blobs, a line search links the rest into tracks, a tree model scores them, a catalogue check removes the known ones. We measure everything on planted fakes and held-out nights; single frames reach magnitude 21.4, shift-and-stack reaches 22.1, and bigger models or more data did not help. No discovery is claimed."

F. Tricky ones

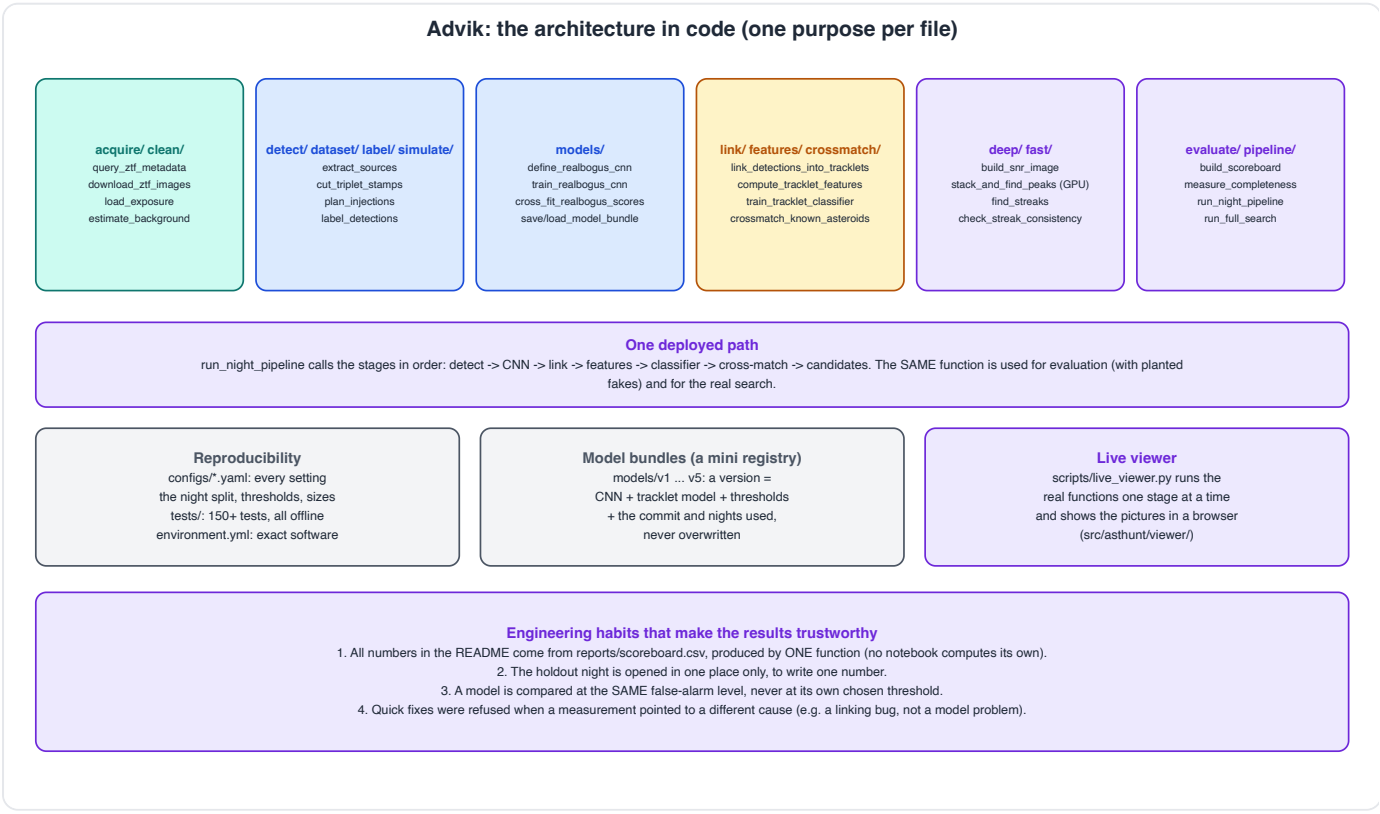
31. "You said honest, but you looked at the test night while fixing bugs." Yes, and we say so. The fixes were physics corrections (coordinates, units, a minimum speed), not tuning to raise a score. To be clean we added a holdout night that only the scoreboard opens; its numbers match validation, which is the evidence nothing was secretly tuned.

32. "Isn't planting fakes circular?" It is circular only if fakes differ from reality. We use the real PSF, noise and frame times, put them on the sky correctly, test realism by adversarial validation (AUC 0.47), and cross-check with real known asteroids: both fade between magnitudes 20.5 and 21.5.

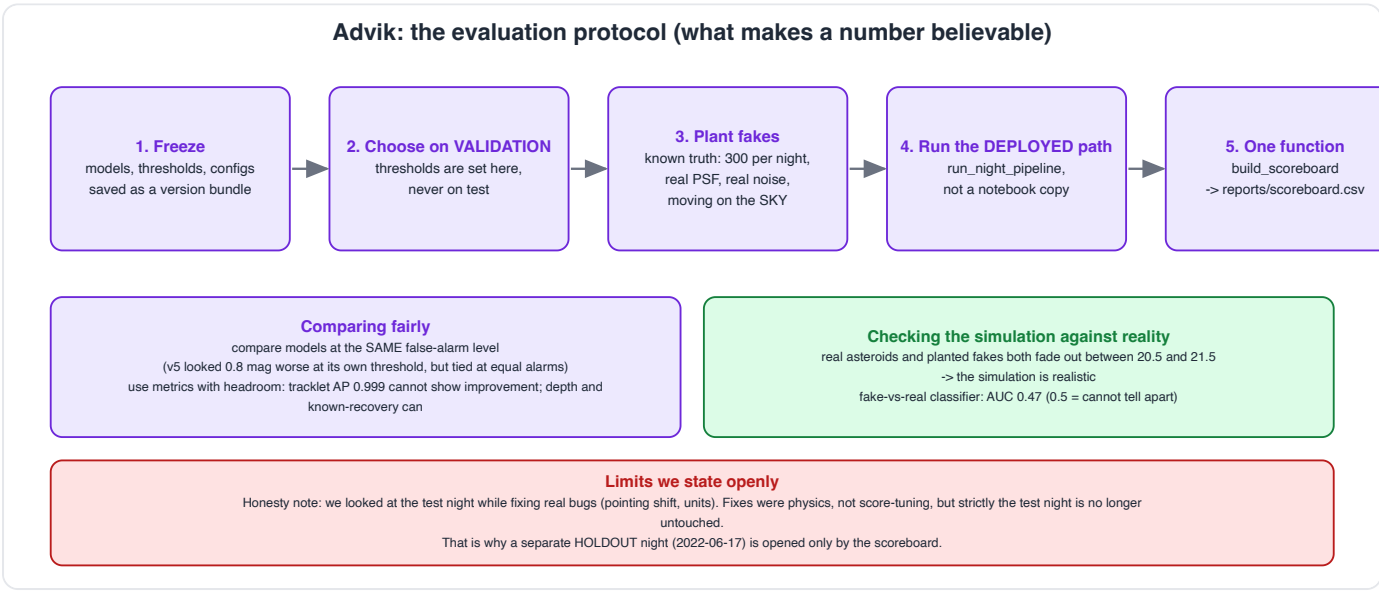
33. "Your CNN got AP 0.938 on test. Is that good?" It beat the feature baseline on every split and cuts false alarms 4.6x at the same recall. But AP on a planted-fake-heavy set flatters; the numbers that matter are depth, known asteroids recovered and unexplained alarms per night.

34. "How much of this did you write yourselves?" (answer truthfully; see `00_Start_Here/07_AI_Use_and_Integrity.md`). State that the project was built with an AI coding assistant under your direction, say which parts you personally ran, changed or extended, and demonstrate understanding by explaining a part without notes.

Diagrams



architecture full



evaluation protocol

Advik: how the live viewer works

Browser page
pick a night, press Start
polls the state every 0.8 s
shows pictures + numbers
Next / Stop buttons

→

Small web server
serve_live_viewer.py
(Python standard library)
/api/state, /img/..
/api/start, /api/next

→

Shared state
live_state.py
stages, numbers, pictures
a gate that waits for
'Next' in step mode

→

Traced run
run_traced_night.py
calls the REAL functions
in the real order and
reports each stage

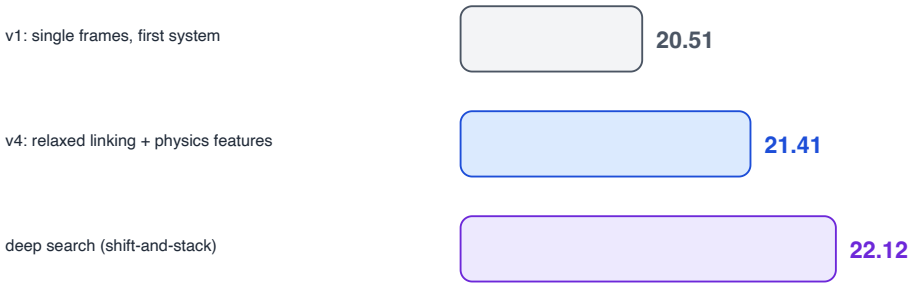
Why you can trust what it shows
Tested: on a real night the traced run gives the SAME detections, scores, tracks and candidates as run_night_pipeline (maximum difference 0.0).
'fakes' plants fake asteroids first, so you can watch them found (bright) or missed (too faint for a single frame).

12 stages: pick night -> open files -> noise -> plant fakes -> find blobs -> stamps -> CNN -> link -> features -> classifier -> known? -> result

live viewer flow

Advik: how deep can we see? (50%-completeness magnitude on the test night 2019-02-08)

Larger magnitude = fainter. Completeness = fraction of planted fakes recovered.
axis starts at magnitude 19



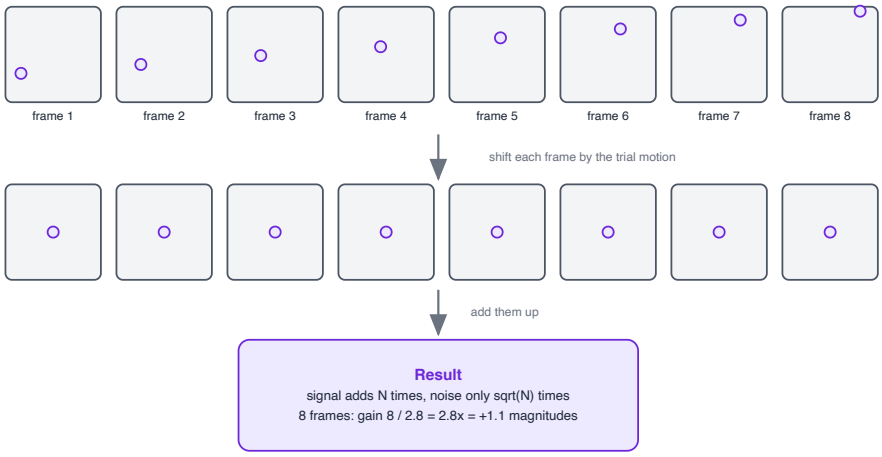
Lesson 1
v1 -> v4: found where asteroids were lost (linking), not by a bigger model.
A multi-frame CNN TIED XGBoost: kept the simpler model.

Lesson 2
v5 (3x more training data) did NOT help at equal false alarms.
The limit is the photons in one 30 s frame, not the model.

scoreboard versions

Advik: shift-and-stack, how we see what no single frame shows

A faint asteroid is too weak in any one 30-second frame. If we shift the frames so the asteroid sits on the same spot and ADD them, it adds up; the noise does not.



Result
signal adds N times, noise only sqrt(N) times
8 frames: gain 8 / 2.8 = 2.8x = +1.1 magnitudes

Search over motions
We do not know the motion, so we
TRY 4,888 trial motions
(every direction, up to 120 arcsec/hour)
on the GPU and keep the best

Picking the best of 4,888 = the
LOOK-ELSEWHERE effect: noise alone
reaches 5 sigma, so the cut is 6 sigma

Making it honest
real difference images are NOT Gaussian:
clip each frame at 5 sigma; remove the
night's median; standardise every pixel;
mask what single frames already see

peaks per stack: 47,550 -> 94
null test (times shuffled): 0 false tracks

shift and stack